

RDC

ChipRun // Digital-design

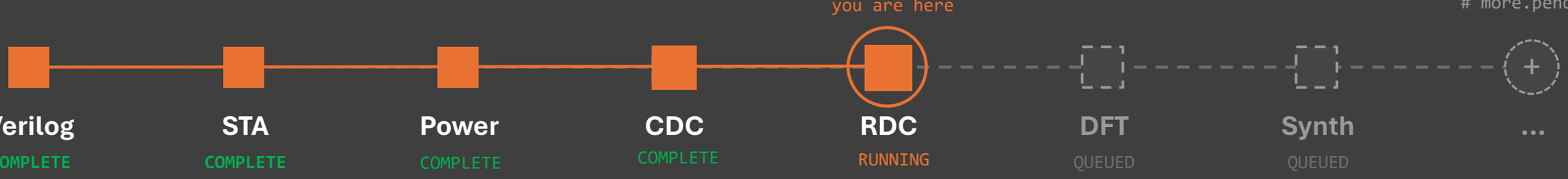
Episode 05 — RDC

Phase 02

It's been a long run,

you are here

more.pending



But We haven't $arrived yet.

GitHub
code examples

Drive
full files and slides

LinkedIn
new episode alerts

Topics

Topics

What We Will Cover

1. Introduction
2. Reset Basics (Recap)
3. The Async Reset Release Problem
4. Reset Synchronizers
5. Reset Domain Crossing
6. Real-World RDC Issues
7. RDC Design Solutions
8. RDC Verification
9. Coding Guidelines and Examples
10. STA and RDC

Introduction

Introduction

What is a Reset?

- **A reset forces the design into a known, valid state.**
    - **Simulation:** the RTL starts from a known state instead of X.
    - **System:** recovers the chip from functional failures — software asserts reset to clear stuck logic and pending errors.
    - **Safety:** fault-tolerant systems trigger reset to recover from transient faults.

- Modern SoCs have many reset sources: Power-on Reset “PoR”, global resets, and per-block functional resets — and that is where RDC begins.

Introduction

Reset Strategy Questions

Before choosing a reset strategy for an ASIC, We should ask:

- Synchronous or asynchronous reset?
- Will every flip-flop receive a reset?
- How will the reset tree be laid out and buffered?
- How do we verify reset timing in STA?
- How do we test reset with scan vectors (DFT)?
- How do we apply reset across multiple clock domains? — the RDC question

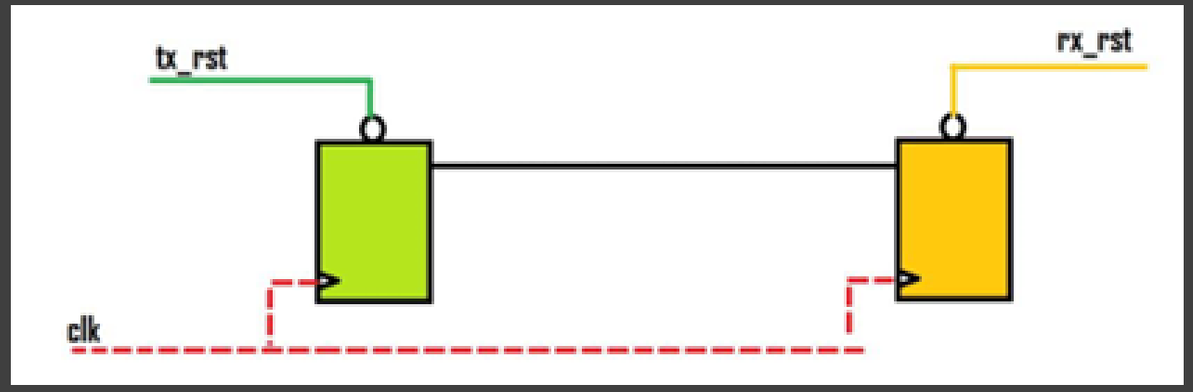
- **It is recommended to make every flop resettable.**

Introduction

CDC vs RDC

- **CDC:** data crosses between **CLOCK** domains — fixed by 2-flop synchronizers and handshakes.
- **RDC:** data crosses between **RESET** domains — synchronizers do **NOT** fix it.

- **Reset domain crossing:** a path whose transmitting flop has an asynchronous reset, and whose receiving flop has a DIFFERENT asynchronous reset — or no reset at all.



- **CDC** sign-off does not look at resets — **RDC** needs its own static and formal verification.

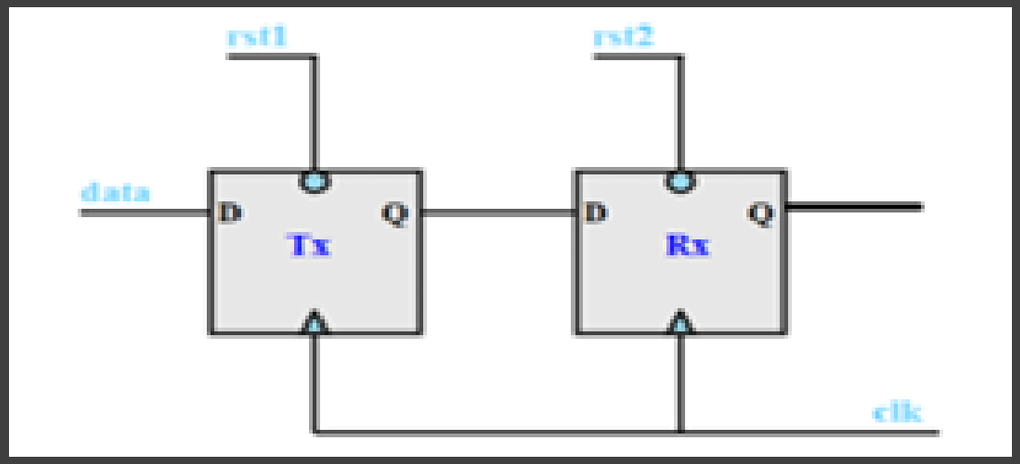
Introduction

Why RDC is a Problem

- The Tx flop reset ASSERTION is an asynchronous event — it acts like an unplanned clock edge.

- If **rst_1** asserts near the **clk** edge, **RX_D** changes inside the Rx setup/hold window -> **RX_Q** can go metastable.

- Metastability spreads garbage downstream, can corrupt the reset state, and in the worst case overheats the device.



Reset Basics (Recap)

Reset Basics (Recap)

Synchronous Reset

- Reset is sampled only on the active clock edge — the reset is NOT in the sensitivity list.
- The if/else order gives the reset priority over the data.
- **Limitation: the reset only works while the clock is running.**

RDC - 00_reset_basics_sync_reset_ff.v

```v
// Synchronous reset flip-flop (Cummings SNUG2002, Example 4a style)
// * Reset is NOT in the sensitivity list -> sampled on the clock edge only
// * Reset gets priority via the if/else order
module sync_reset_ff (
    output reg q,
    input  d,
    input  clk,
    input  rst_n     // synchronous, active-low
);
    always @(posedge clk)
        if (!rst_n) q <= 1'b0;
        else        q <= d;
endmodule
```

Reset Basics (Recap)

Synchronous Reset — Pros and Cons

- **Advantages:**
  - smaller flops; the circuit stays 100% synchronous
  - the clock filters small reset glitches
  - flops can help buffer the reset tree

- **Disadvantages:**
  - needs a pulse wider than one clock period (pulse stretcher)
  - can be masked by X in simulation
  - cannot reset without a clock (power-up!)
  - adds gates in the data path “considered as a data-path signal not a reset pin in synthesis”.

- **Note:** *glitches near the active clock edge can still cause metastability — the clock is not a perfect filter.*

Reset Basics (Recap)

Asynchronous Reset

- Reset is in the sensitivity list.
- Takes effect immediately — with or without a clock.
- Synthesis maps it directly to a flop with an **async reset pin** — no data-path logic.

RDC - 01_reset_basics_async_reset_ff.v

```v
// Asynchronous reset flip-flop (Cummings SNUG2002, Example 5a style)
// * Reset IS in the sensitivity list -> takes effect immediately,
//   with or without a clock
// * Reset still has priority over the clock in the if/else
module async_reset_ff (
    output reg q,
    input  d,
    input  clk,
    input  rst_n     // asynchronous, active-low
);
    // Verilog-2001 also allows: always @(posedge clk, negedge rst_n)
    always @(posedge clk or negedge rst_n)
        if (!rst_n) q <= 1'b0;
        else        q <= d;
endmodule

```

Reset Basics (Recap)

Asynchronous Reset — Pros and Cons

- **Advantages:**
  - guaranteed clean data path
  - resets without a clock
  - automatic synthesis mapping to async-reset flops.

- **Disadvantages:**
  - reset **removal** near a clock edge can cause metastability “recovery and removal times”
  - board noise can cause spurious resets; must be masked for DFT scan.

Reset Basics (Recap)

Sync vs Async — Summary

Neither style is free — pick per design, then apply the guidelines of this deck.

	Synchronous	Asynchronous
Data path	reset logic in the D-path	clean — reset pin on the flop
Needs clock to reset	Yes	No
Glitch filtering	clock filters most glitches	none — any wide-enough pulse resets
Reset removal	already synchronous	must be synchronized
Flop area	smaller	slightly larger

Reset Basics (Recap)

Coding Style: One FF Type per Block

- Do not mix resetable flops with follower (non-reset) flops in one always block.
- With a synchronous reset, rst_n becomes a data-enable -> a useless loadable flop is synthesized.

RDC - 02_reset_basics_bad_follower_style.v

```
1  // BAD: a resetable flop mixed with a follower (non-reset) flop
2  // inside ONE always block (Cummings SNUG2002, Example 1a).
3  // With a synchronous reset, rst_n becomes a data-enable for the
4  // second flop -> synthesizes an unnecessary loadable flop.
5  module bad_follower_style (
6      output reg q2,
7      input  d,
8      input  clk,
9      input  rst_n
10 );
11     reg q1;
12     always @(posedge clk)
13         if (!rst_n) q1 <= 1'b0;
14         else begin
15             q1 <= d;
16             q2 <= q1;   // follower gets a hidden load-enable!
17         end
18 endmodule
19
```

RDC - 03_reset_basics_good_follower_style.v

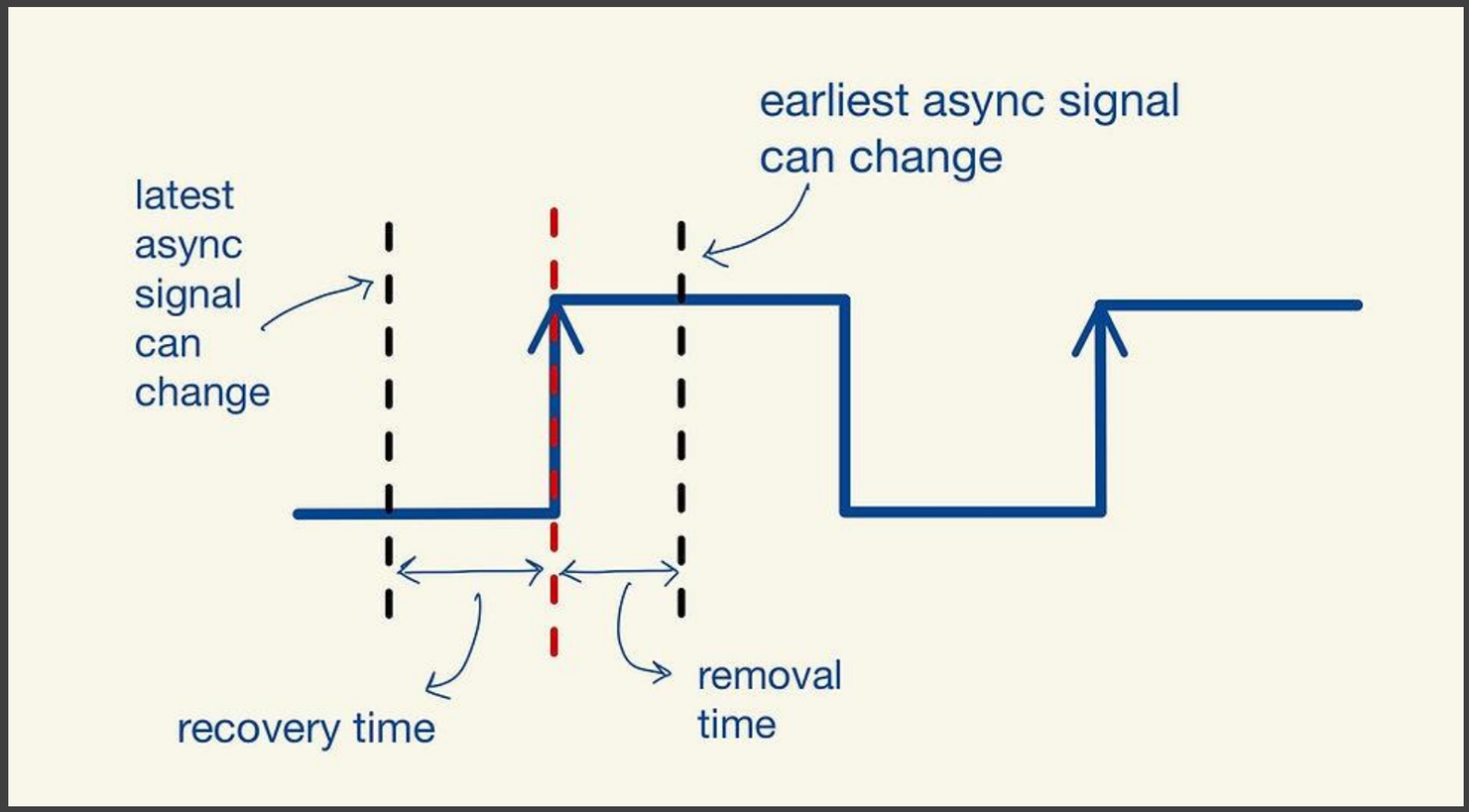
```
1  // GOOD: two separate always blocks -> one flop type per block
2  // (Cummings SNUG2002, Example 2a). The follower stays a clean flop.
3  module good_follower_style (
4      output reg q2,
5      input  d,
6      input  clk,
7      input  rst_n
8  );
9      reg q1;
10     always @(posedge clk)
11         if (!rst_n) q1 <= 1'b0;
12         else        q1 <= d;
13
14     always @(posedge clk)
15         q2 <= q1;       // follower: no reset, no enable
16 endmodule
17
```

The Async Reset Release Problem

The Async Reset Release Problem

Assertion is Easy — Removal is Hard

- **Assertion:** asynchronous, immediate, no timing requirement — safe.
- **Release:** also asynchronous -> two failure modes:
  - **Recovery:** reset must be DE-asserted at least the recovery time **BEFORE** the active clock edge (like setup).
  - **Removal:** reset must stay asserted at least the removal time **AFTER** the clock edge (like hold).

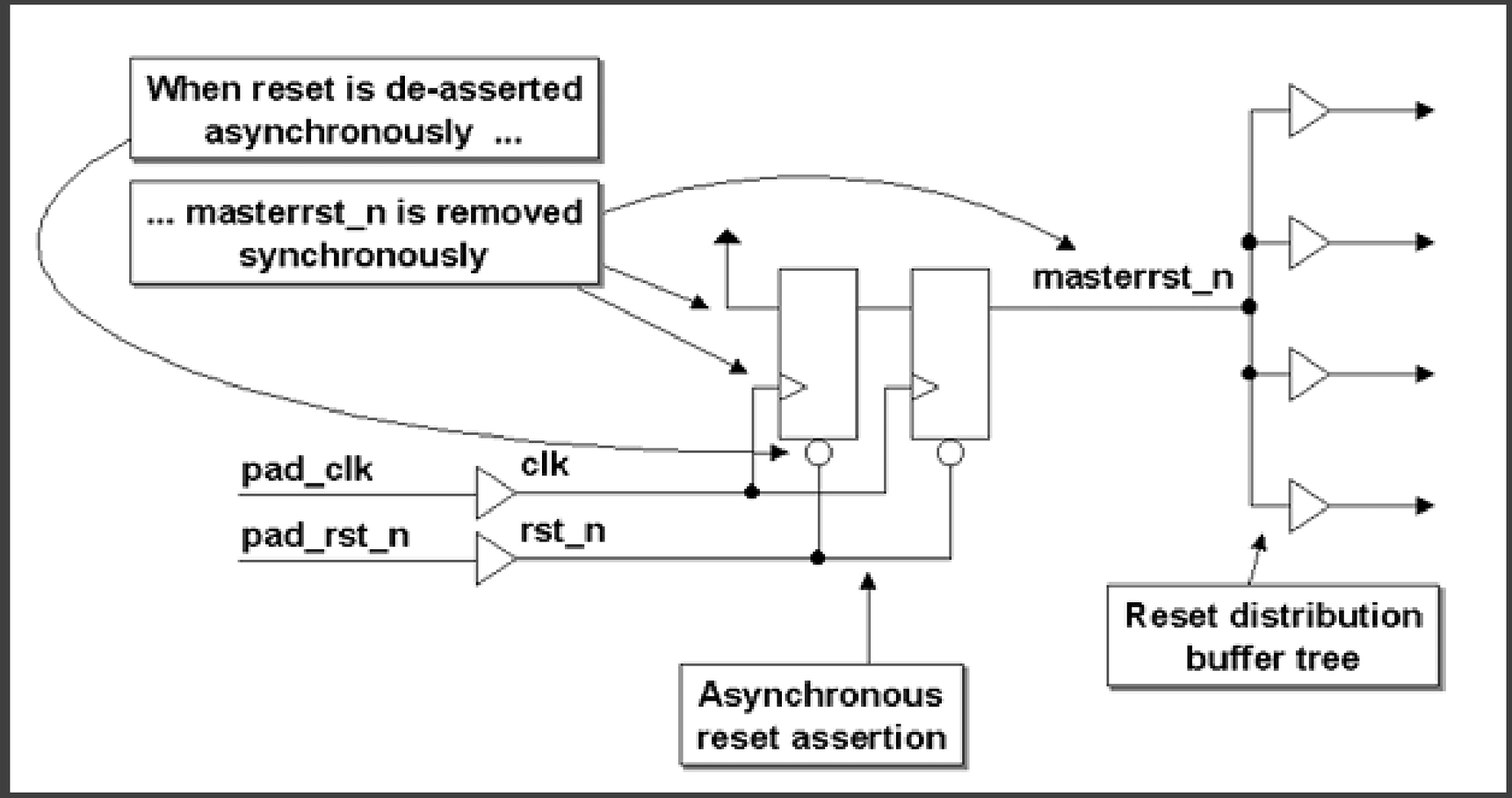


Reset Synchronizers

Reset Synchronizers

Async Assert, Sync Release

- **EVERY design using an asynchronous reset must include a reset synchronizer.**
- Raw reset asynchronously drives two flops whose D inputs are tied high.
- The assertion of the rst happen asynchronously with the masterrst.
- The removal happens through two clock edges while the second FF present to remove any metastability that might be caused by the reset signal being removed asynchronously and too close to the rising clock edge.



Reset Synchronizers

Reset Synchronizer — Code

- Both flops reset asynchronously; on release a 1 shifts through:
- rst_n rises exactly 2 clk edges after arst_n releases — glitch-free.

RDC - 04_reset_synchronizer.v

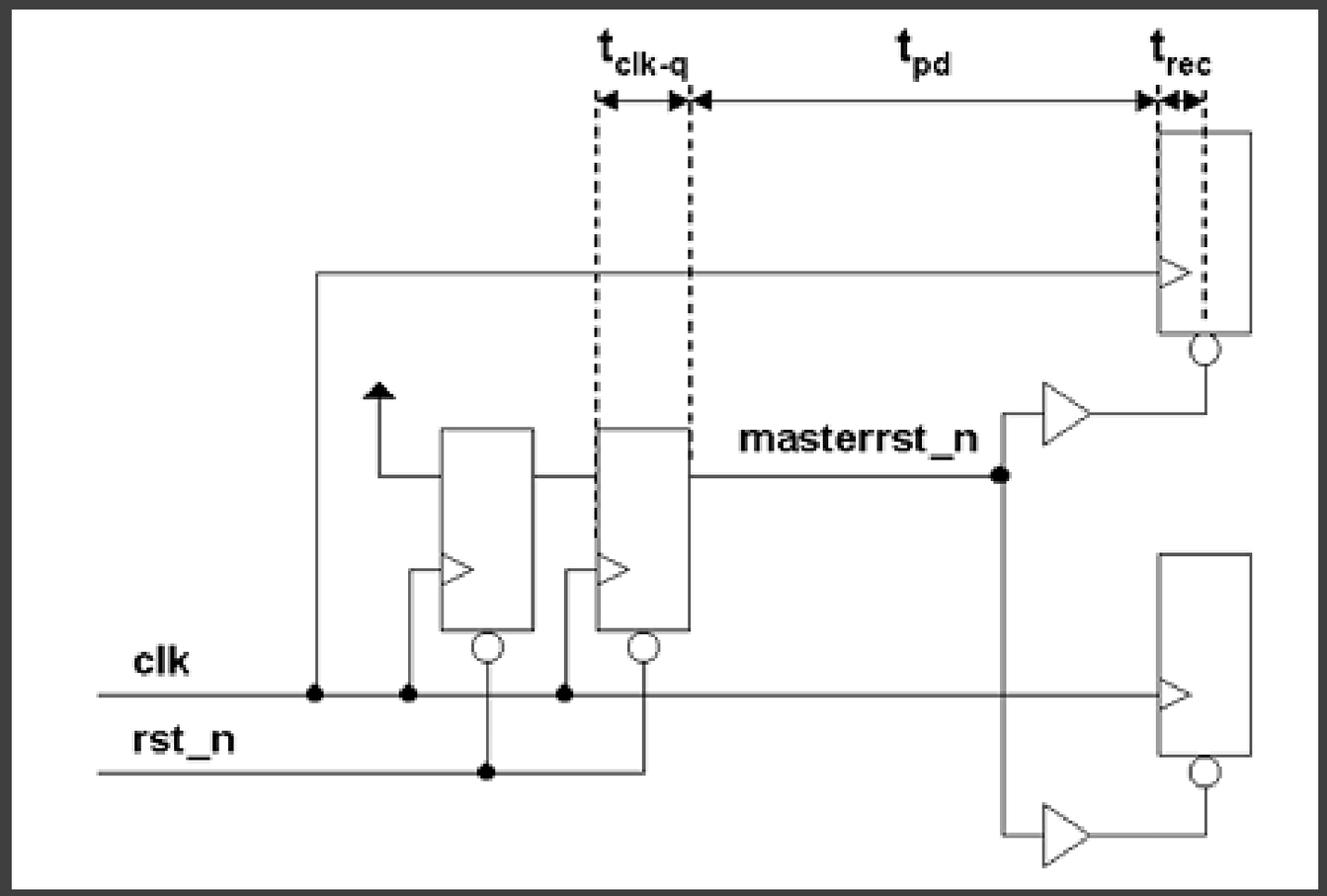
```v
// Reset synchronizer: asynchronous assert, SYNCHRONOUS release
// * arst_n : raw async reset (pad / PoR)
// * rst_n  : release aligned to clk rising edges -> meets recovery time
module reset_synchronizer (
    output reg rst_n,
    input  clk,
    input  arst_n
);
    reg rff1;
    always @(posedge clk or negedge arst_n)
        if (!arst_n) {rst_n, rff1} <= 2'b00;
        else         {rst_n, rff1} <= {rff1, 1'b1};
endmodule

```

Reset Synchronizers

Reset Removal Timing Budget

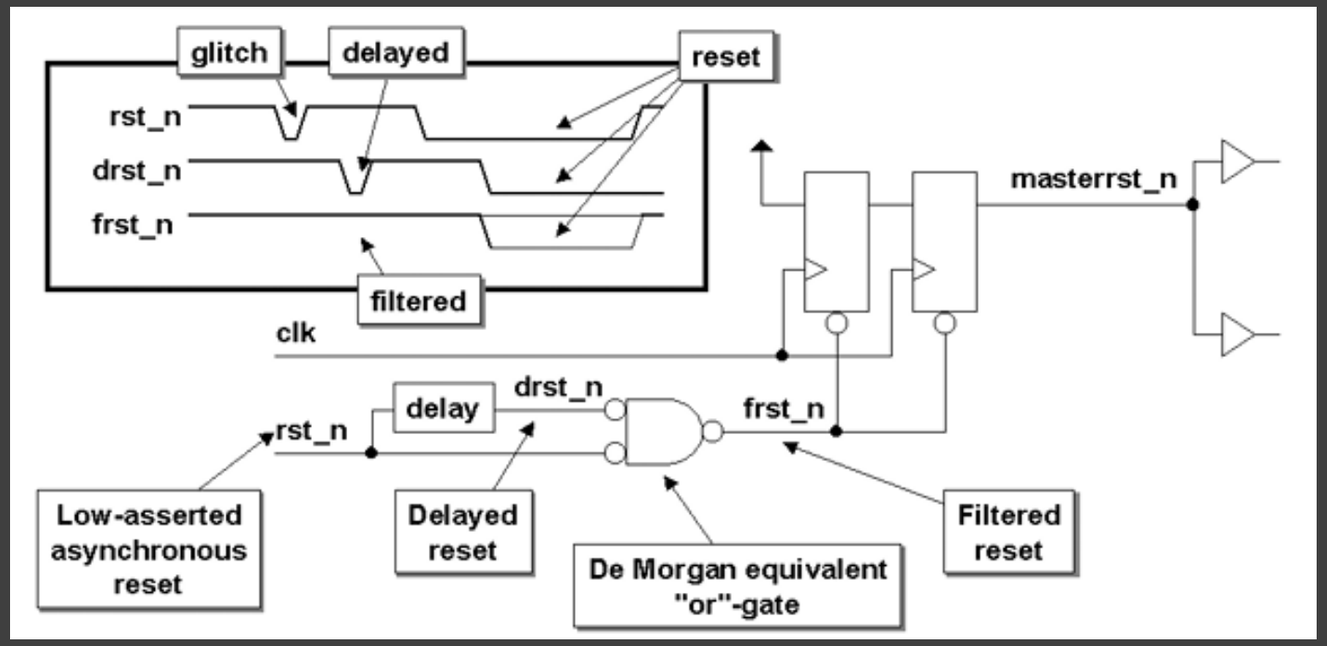
- **Tclk-to-Q(synchronizer) + T(reset tree) + Trecovery(destination) < Tclk**
- This is why the reset tree is timed in STA after layout — treat it like a second clock tree.
- Clocking the reset flop from a leaf clock makes the budget easier to close.



Reset Synchronizers

Reset Glitch Filtering

- An async reset responds to ANY pulse wide enough to meet its pulse width — board noise becomes a reset.
- **Fix:** delay the assertion through a small delay line; glitches shorter than the delay vanish.
- Only needed where the reset source is noisy — and it needs a running clock.



RDC - 15_reset_glitch_filter.v

```
// Reset glitch filter.
// An async reset answers ANY pulse wide enough to trip the flop, so
// board noise becomes a spurious reset. Delay the ASSERTION through a
// WIDTH-stage shift register: a glitch shorter than WIDTH clk cycles
// never reaches the downstream synchronizer.
// * needs a free-running clock (that is why Cummings calls it "ugly")
// * add a Schmitt-trigger pad on the raw reset input
// * feed rst_flt_n into a normal reset synchronizer afterwards
module reset_glitch_filter #(parameter WIDTH = 4) (
    output rst_flt_n,
    input  raw_rst_n,   // noisy async reset input
    input  clk          // free-running clock required for the filter
);
    reg [WIDTH-1:0] dly;
    initial begin
        dly = {WIDTH{1'b1}};   // FPGA sim; on ASIC the PoR covers startup
    end
    always @(posedge clk)
        dly <= {dly[WIDTH-2:0], raw_rst_n};
    // low only after raw_rst_n stayed low for WIDTH clk edges
    assign rst_flt_n = dly[WIDTH-1];
endmodule

```

Reset Domain Crossing

Reset Domain Crossing

RDC — Formal Definition

- **RDC path: Tx flop has an async reset AND (Rx flop has a different async reset OR Rx has no reset).**
- The crossing itself is not automatically a bug.
  - **Unsafe:** metastability can propagate — Tx async assertion can hit the Rx setup/hold window.
  - **Safe:** no metastability can propagate (by ordering, isolation, or timing).

- **Same reset domain does not automatically mean safe “removal/recovery”; different domains do not automatically mean broken “RDC can be solved”.**

Reset Domain Crossing

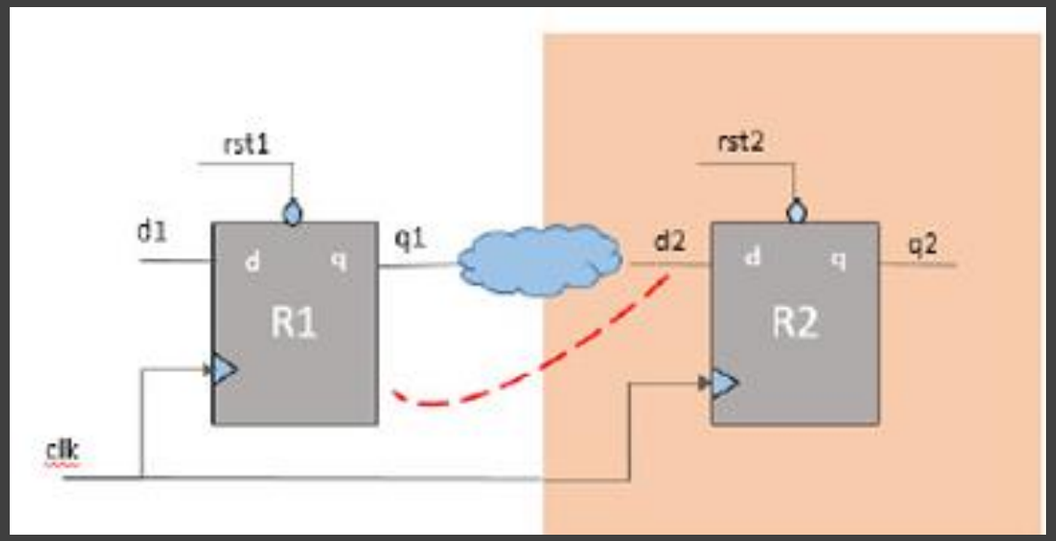
How an RDC Violation Happens

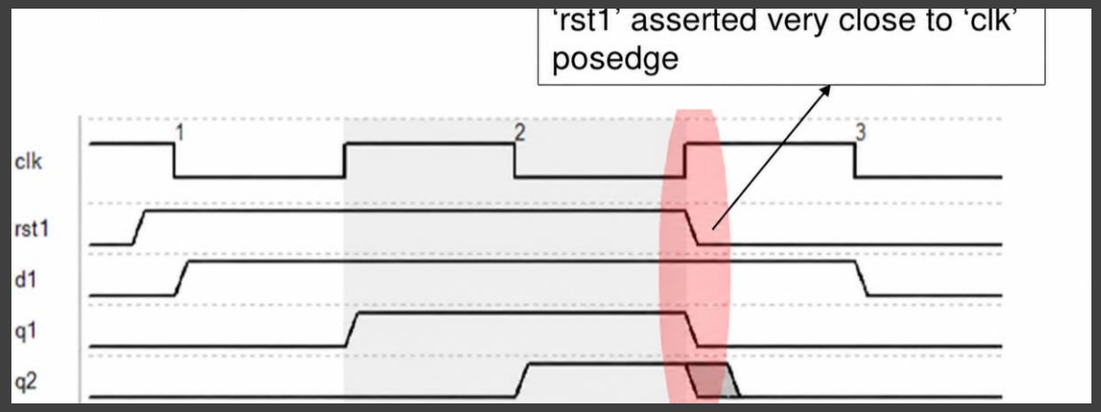
1. rst1 asserts at a random phase of clk (async!).

2. The Tx flop output is forced to its reset value at that moment.

3. The change lands in the Rx flop setup/hold window.

4. The Rx flop goes metastable -> garbage propagates.

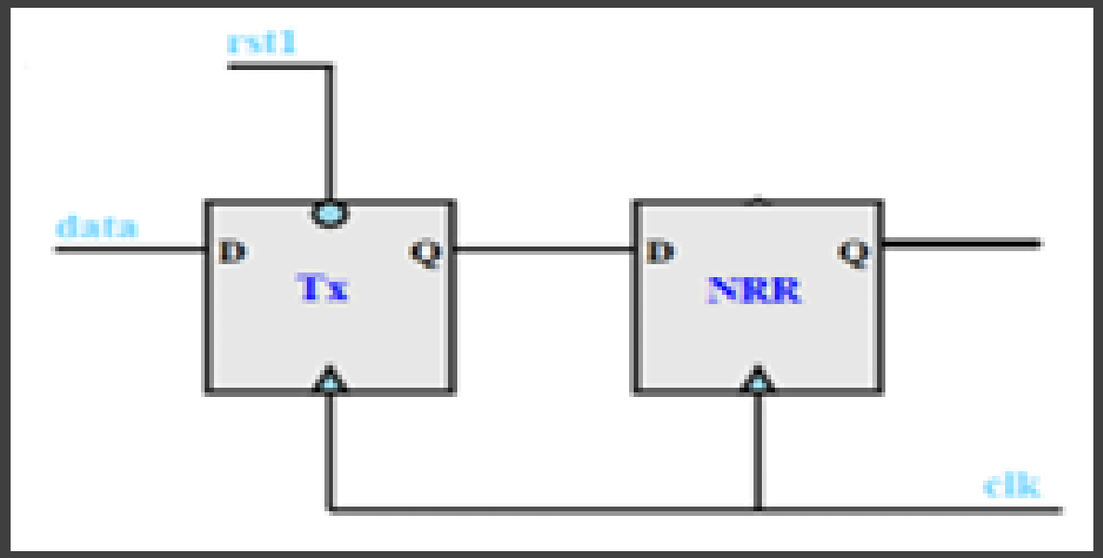




Reset Domain Crossing

Crossing to a Non-Resettable Flop

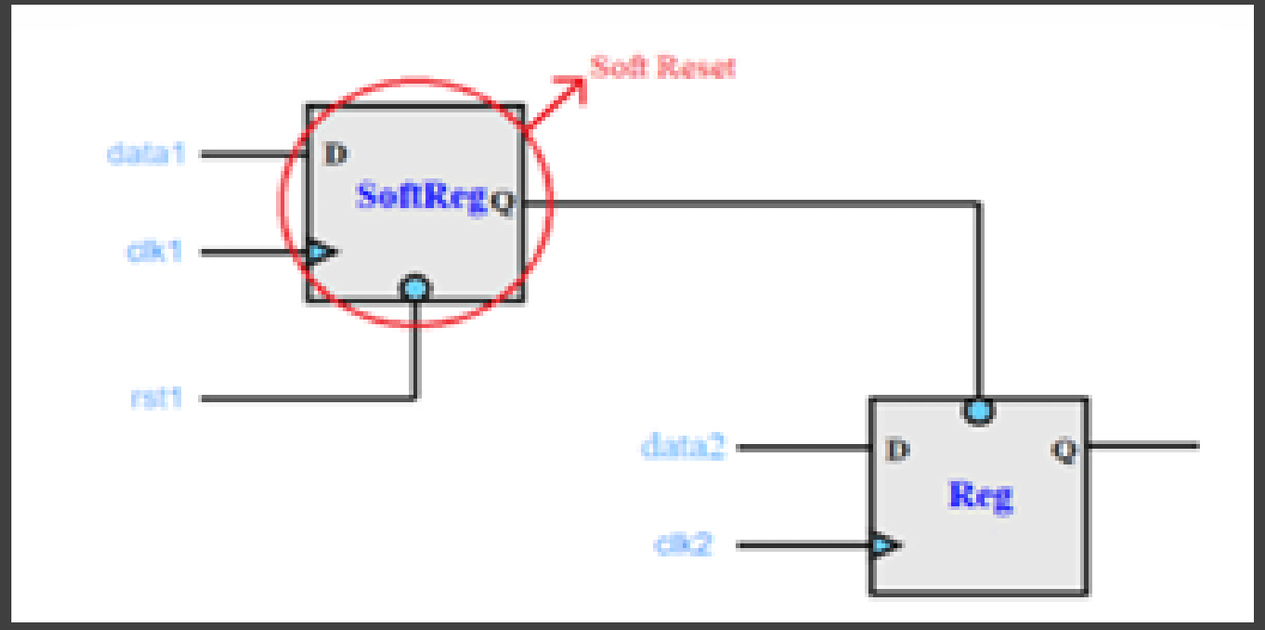
- NRR (non-resettable register): pipeline and follower flops with no reset pin.
- When the Tx reset asserts, the NRR keeps clocking while its input changes -> setup violation -> metastability.
- **Async-to-async is not the only case: async -> no-reset is a violation too.**



Reset Domain Crossing

Soft Resets

- **Soft reset:** an internally generated reset — a register/latch output used as the reset of other flops.
- Used to reset one subsystem without resetting the chip: power saving, fast fault recovery, debug isolation.



RDC - 08_soft_reset_basic.v

```
// Soft reset: a REGISTER output used as the reset of other flops
// SoftReg_n (clk1 domain) is the soft reset of the Reg flop (clk2
// domain): lets you reset one subsystem without resetting the chip.
// NOTE: this exact pattern is an RDC risk when clk1 != clk2.
module soft_reset_basic (
    output reg reg_q,
    input  data1, clk1,
    input  data2, clk2,
    input  rst1_n                  // primary async reset
);
    reg SoftReg_n;
    always @(posedge clk1 or negedge rst1_n)
        if (!rst1_n) SoftReg_n <= 1'b0;
        else         SoftReg_n <= data1;

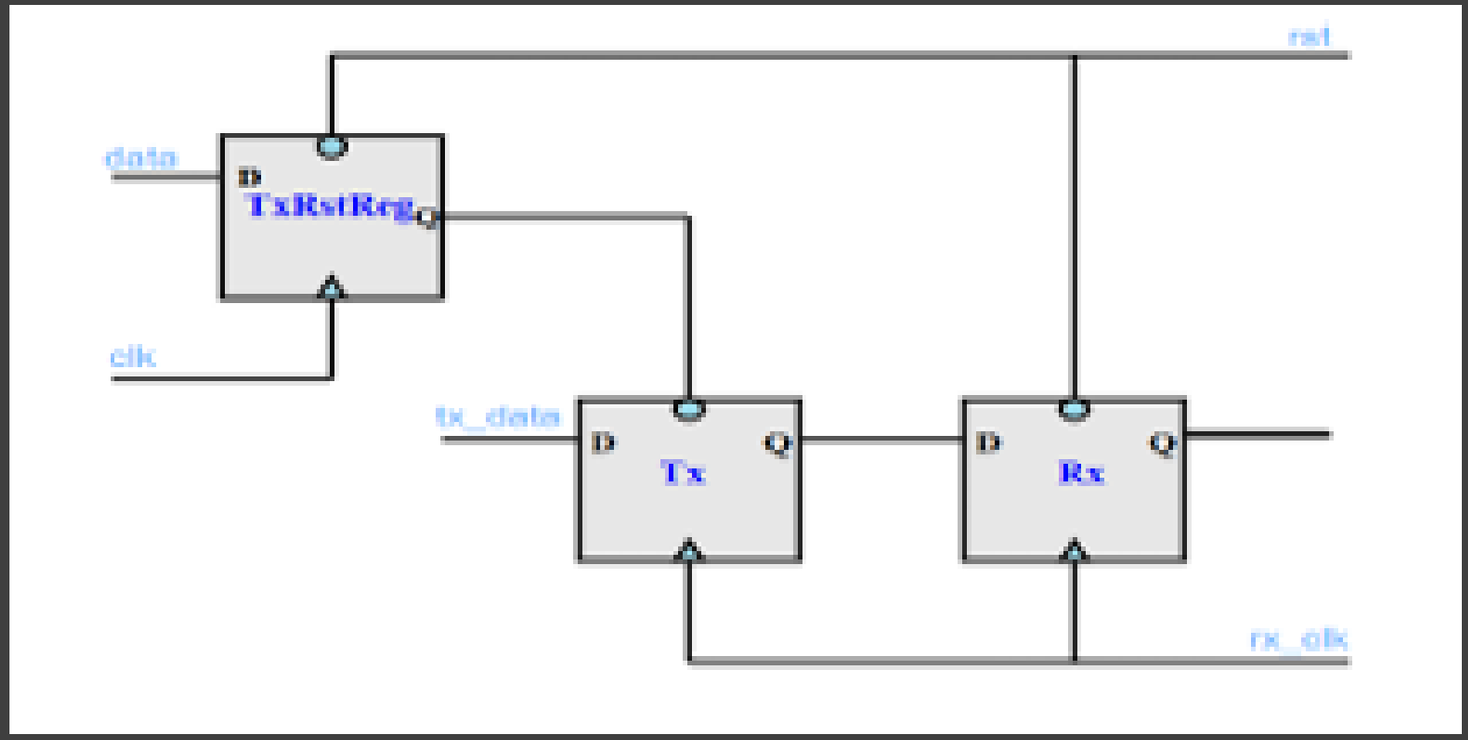
    always @(posedge clk2 or negedge SoftReg_n)   // soft reset in action
        if (!SoftReg_n) reg_q <= 1'b0;
        else            reg_q <= data2;
endmodule

```

Reset Domain Crossing

Soft Reset RDC — Different Clock: UNSAFE

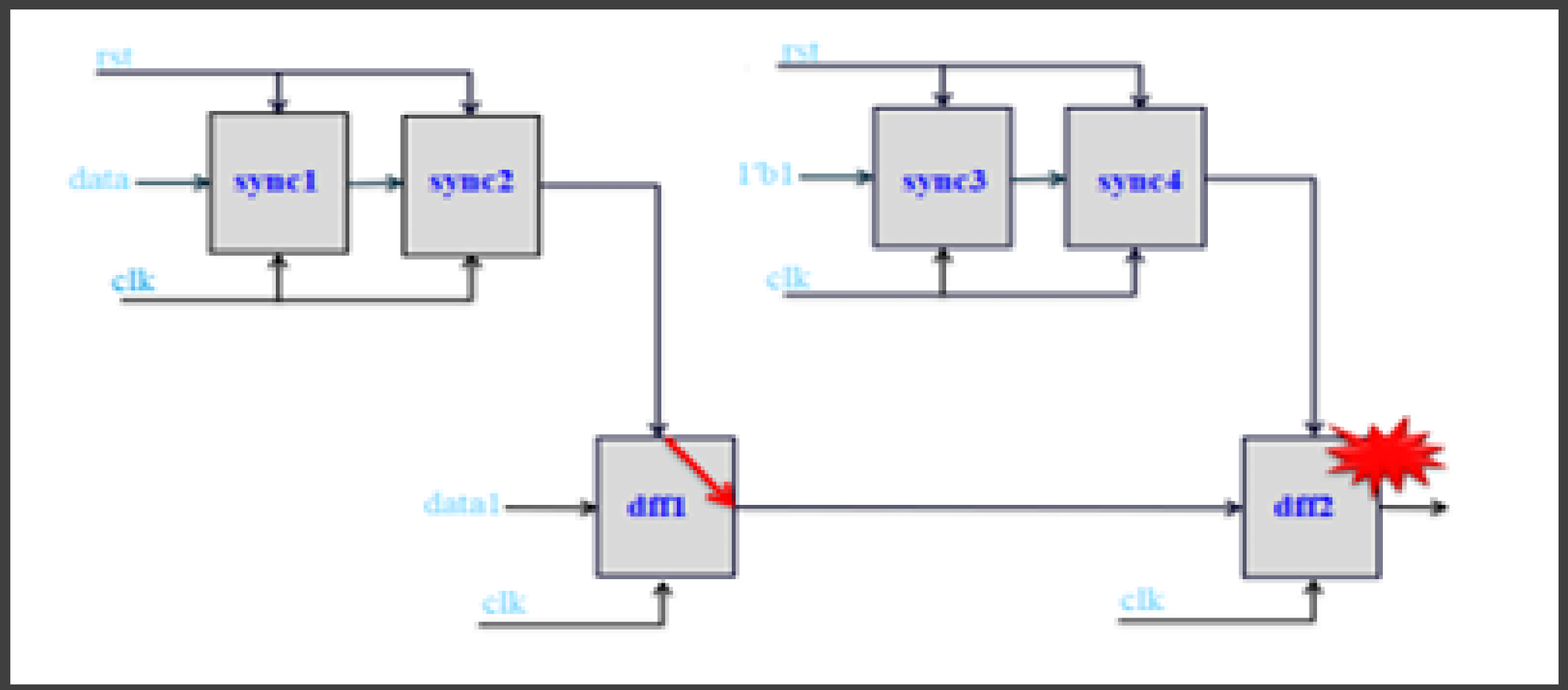
- Tx and Rx share async domain rst — but Tx is reset by a soft reset generated in ANOTHER clock domain.
- soft_n asserts asynchronous to rx_clk -> Tx output changes near the rx_clk edge -> Rx metastable.
- **A shared async reset domain does NOT save you.**



Reset Domain Crossing

Soft Reset RDC — Same Clock: Safe IF STA

- Soft reset generated in the SAME clock domain as both flops -> assertion is synchronous to the clock.
- **Path to time:** TxRstReg/Q -> Tx reset pin -> Tx/Q -> Rx/D as a normal SETUP check.
- If total delay < Tclk - Tsetup -> safe. RDC tools flag it as a CAUTION for STA.



Reset Domain Crossing

RDC Safety Cheat Sheet

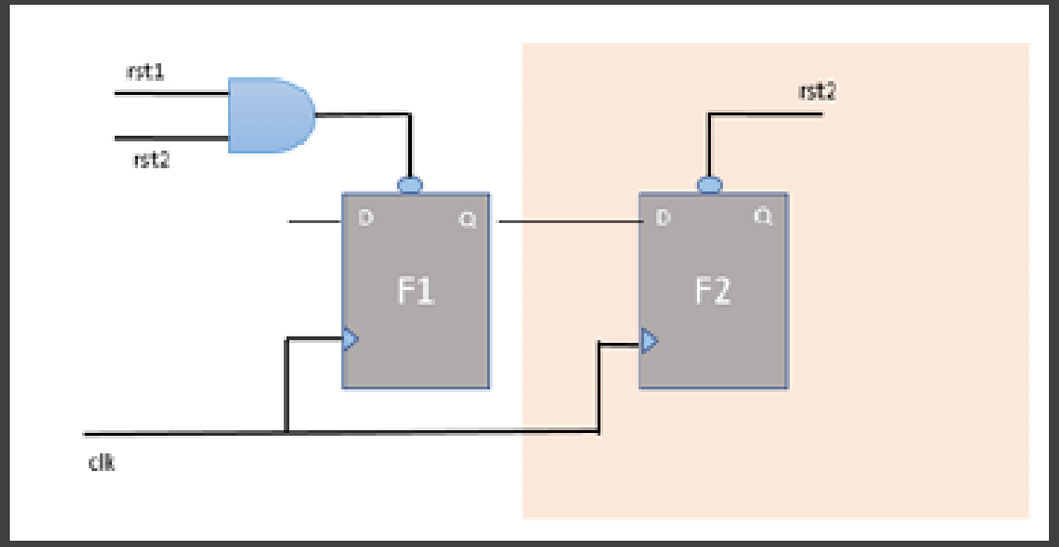
Scenario	Verdict
Different async domains, no soft reset	Violation — fix in RTL
Same async domain, Tx soft reset from a different clock	Violation — fix in RTL
Same async domain, Tx soft reset from the same clock	Caution — must pass STA setup

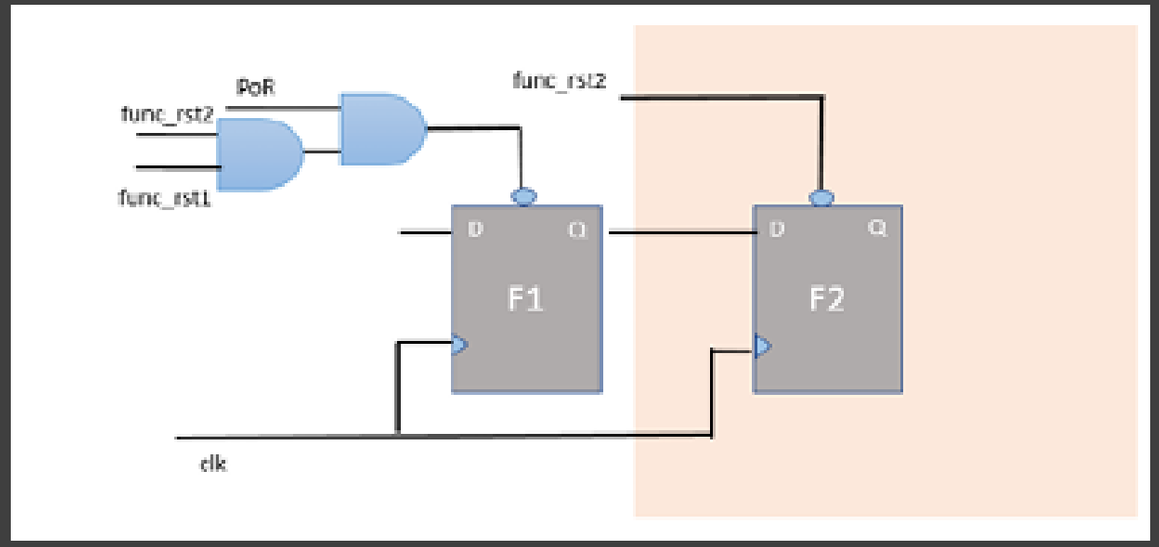
Real-World RDC Issues

Real-World RDC Issues

Combined Resets on Tx or Rx

- Tx flop reset = AND of two resets (rst1 & rst2); or functional reset combined with PoR.
- If the first term asserts before the second -> Tx output changes while Rx is out of reset -> metastability.
- **Every combination of reset sources creates its own ordering requirement — analyze combinations, not just nets.**

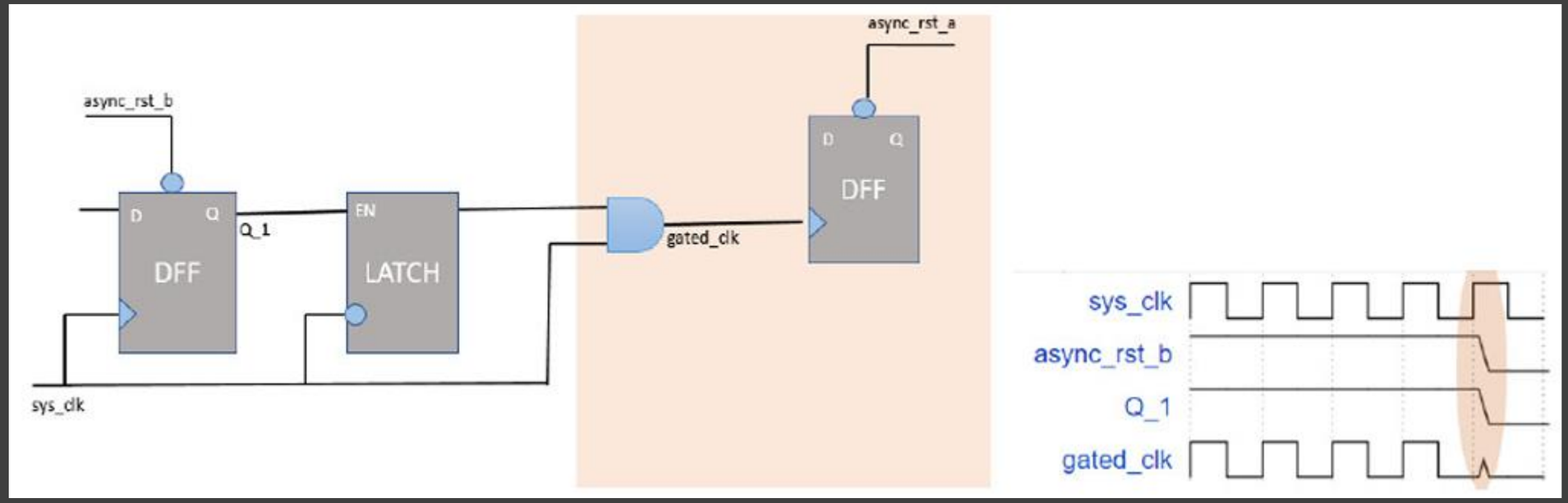




Real-World RDC Issues

Gated-Clock Glitch Across Reset Domains

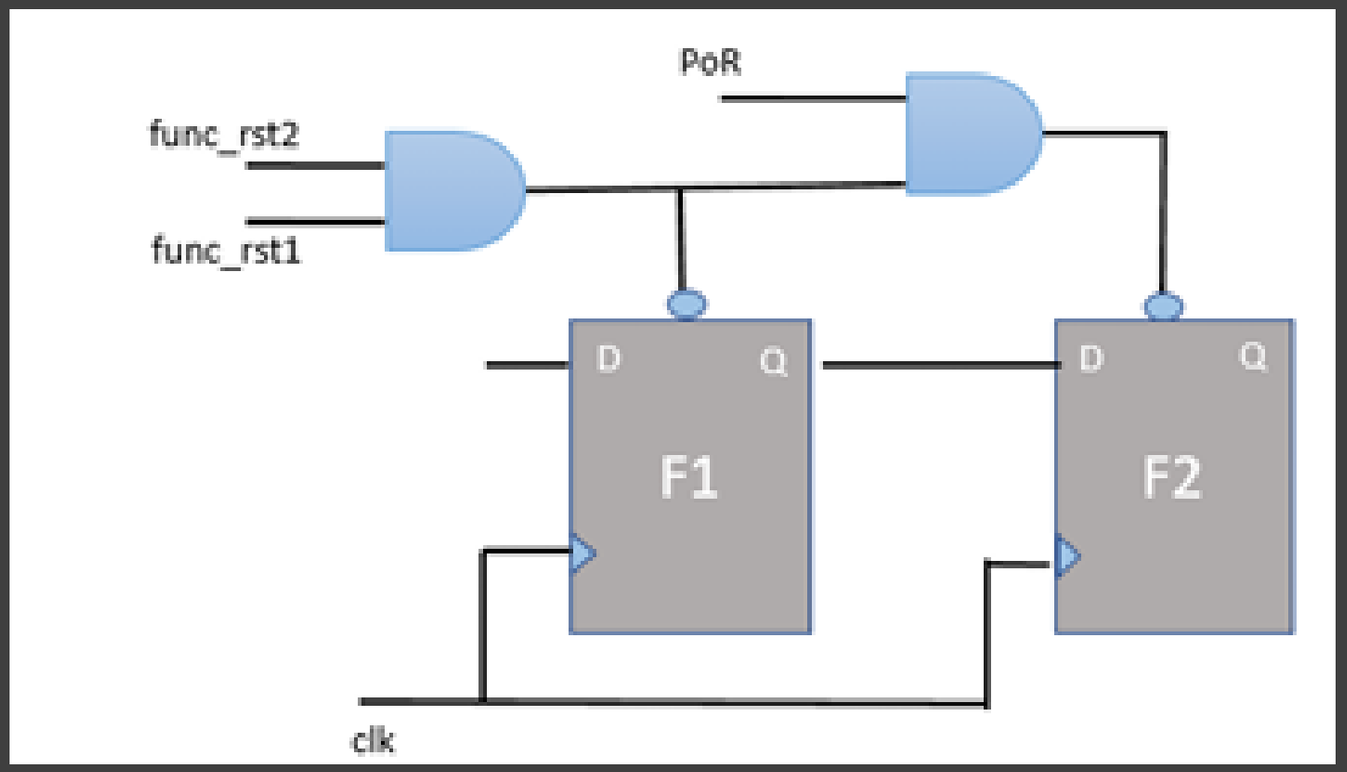
- Q_1 (reset by async_rst_b) enables an ICG clocking a flop reset by async_rst_a.
- async_rst_b asserts while sys_clk is high -> the latch is transparent -> EN falls mid-pulse -> GLITCH on the gated clock.
- The downstream flop (different reset domain) sees a bogus edge.
- **A reset crossing can corrupt the CLOCK network, not just data paths.**



Real-World RDC Issues

Looks Like RDC — Actually Safe

- **Common ancestor:** Tx reset assertion always asserts the Rx reset too — Rx is in reset before any Tx change matters.
- Good RDC tools recognize these dependencies instead of reporting false crossings.

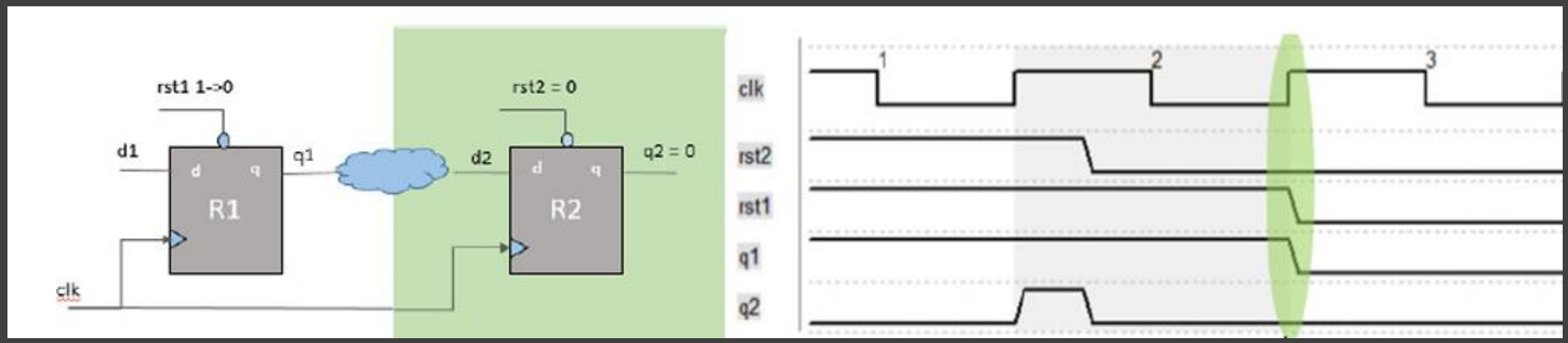


RDC Design Solutions

RDC Design Solutions

Solution 1 — Reset Sequencing

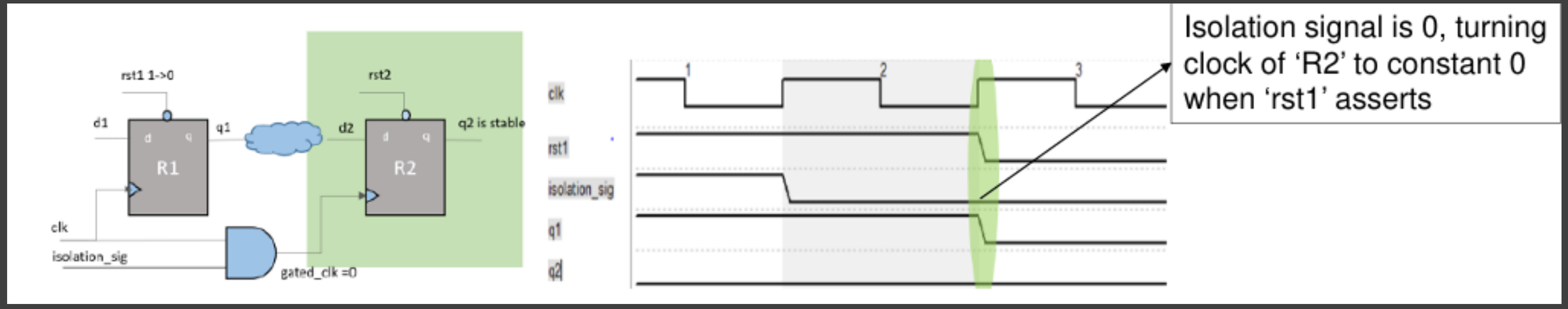
- **Guarantee:** the Rx async reset ALWAYS asserts before (or with) the Tx async reset.
- Rx is already in reset -> any change on its D-pin is harmless.
- Implement with ordered reset synchronizers: the a-domain master reset feeds the b-domain synchronizer input.



RDC Design Solutions

Solution 2 — Clock-Gate Isolation

- Turn OFF the Rx clock before the Tx reset asserts (an isolation enable disables the ICG).
- No clock edge -> no setup/hold sampling -> no metastability.
- Declare the isolation enable and its sequencing as constraints so the tool can prove it.



RDC Design Solutions

Solution 3 — Data Isolation

- Block the Tx -> Rx wire before the Tx reset asserts (AND gate / isolation cell on the path).
- The Rx D-pin holds a stable value through the whole reset event.
- Same idea as UPF isolation — but sequenced by reset ordering, not power states.

RDC Design Solutions

Choosing a Technique

- **Need guaranteed functional recovery?** reset sequencing.
- **Rx island is clock-gated?** clock-gate isolation.
- **Cannot touch the clock?** data isolation.
- **Same-clock crossing anyway?** time it in STA (Caution path).
- **Architectural freedom?** re-partition reset domains — remove the crossing.

- Often two techniques are combined: ordering + isolation on the critical paths.

RDC Verification

RDC Verification

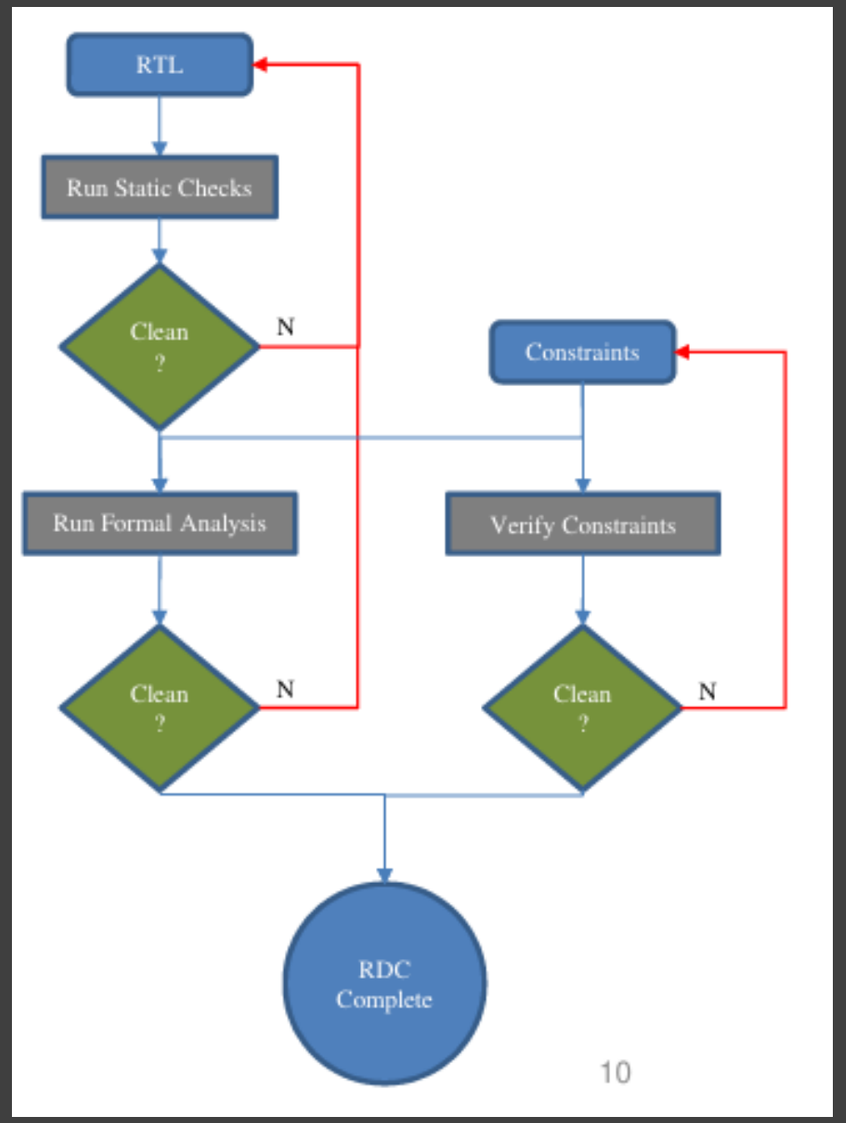
Why Not Just Simulation?

- The violating corner — reset assertion inside the setup/hold window — is a few-picoseconds window; random simulation almost never hits it.
- Gate-level sims with realistic reset trees are slow, and still not exhaustive.
- **Static + formal RDC analysis checks ALL flop pairs across ALL reset scenarios — the CDC-style sign-off for resets.**

RDC Verification

RDC Verification Flow

- Static reset checks gate the flow; then formal analysis and constraint verification must both be clean.
- Each failed gate loops back to RTL or constraints until clean.



RDC Verification

Static Reset Checks

- **Dual synchronicity:** a reset used synchronously on some flops, asynchronously on others.
- **Dual polarity:** active-high on some flops, active-low on others.
- **Async set/reset reused as data:** the reset state cannot be controlled.
- **Glitch-prone reset drivers:** tristate outputs, or resets gated by XOR/XNOR/NAND/NOR logic.
- These are caught before RDC analysis — they are not usually caught by lint.

RDC Verification

Clock, Reset and Soft-Reset Detection

- User specifies primary clocks and resets; the tool traces down to leaf clocks and localized resets.
- Detected resets are classified synchronous / asynchronous / dual and mapped into a reset TREE (root = primary source).
- **Soft resets (flop/latch outputs driving reset pins) are detected automatically — they create the hidden crossings.**

RDC Verification

Constraints

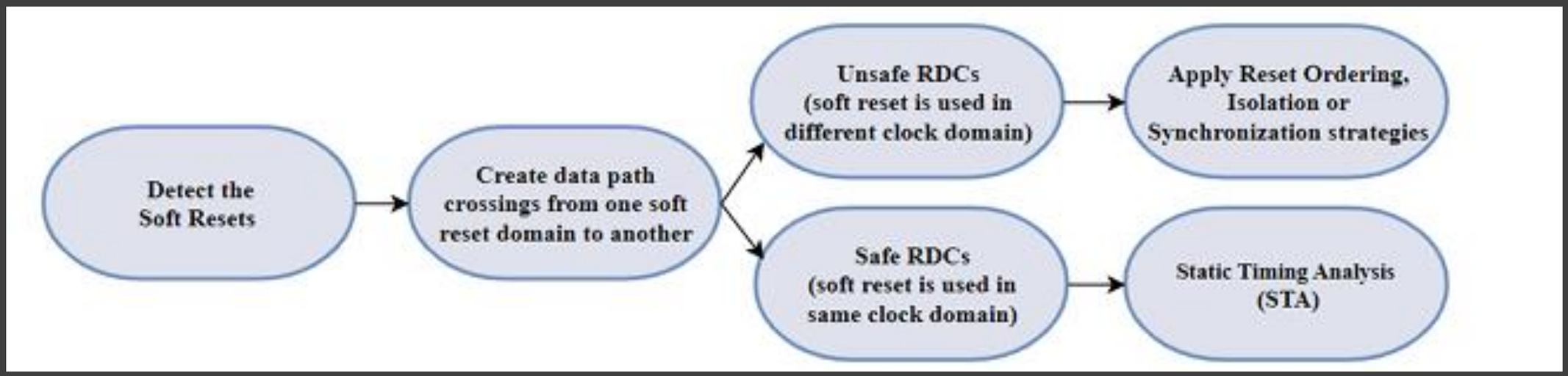
- Declare clocks and resets the tool cannot infer (black boxes, gated clocks).
- **Reset sequencing:** which reset asserts first — proves crossings safe.
- **Isolation enables:** validated by functional/formal checks.

- **Constraint quality = result quality: fewer false crossings, faster closure.**

RDC Verification

Analyze, Fix, Re-Verify

- **Violations:** restructure, synchronize, isolate, or re-order the resets.
- **Cautions:** hand to STA — time the reset path as setup.
- Re-run RDC after every fix; track crossings to closure.



RDC Coding Guidelines

RDC Coding Guidelines

Guidelines — Reset Architecture

- Every async reset enters the chip through a reset synchronizer.
- One synchronizer + one reset tree per clock domain.
- Order the resets of domains that talk to each other **(Rx asserts first)**.
- Never generate an async reset from combinational logic — **glitch risk**.
- Soft resets: generated in the **same clock domain** that consumes them.
- Keep reset polarity consistent per net; **never** reuse a reset net as data.

RDC Coding Guidelines

Guidelines — RTL

- One flop type per always block; nonblocking (<=) in sequential logic.
- Give every flop a reset **unless** it is a deliberate follower.
- Do not gate an async reset with logic on its way to the reset pin — **glitch risk**.
- Instantiate reset synchronizers explicitly; protect them from synthesis (dont_touch).
- Never let a reset from one domain gate the clock of another domain.

RDC Coding Guidelines

Example — Unsafe Crossing

- **Tx reset by rst_a_n, Rx reset by rst_b_n — the classic Violation.**
- An RDC tool flags the path q_a -> b_reg; fix it with one of the next techniques.

RDC - 06_rdc_unsafe_crossing.v

```v
// UNSAFE RDC: Tx flop reset by rst_a_n, Rx flop reset by rst_b_n.
// rst_a_n can assert close to a clk_b rising edge -> q_a changes
// inside the Rx setup/hold window -> b_reg goes metastable.
module rdc_unsafe_crossing (
    output reg b_reg,
    input  d_a,
    input  clk_a, clk_b,
    input  rst_a_n, rst_b_n      // two independent async resets
);
    reg q_a;
    always @(posedge clk_a or negedge rst_a_n)
        if (!rst_a_n) q_a <= 1'b0;
        else          q_a <= d_a;

    always @(posedge clk_b or negedge rst_b_n)
        if (!rst_b_n) b_reg <= 1'b0;
        else          b_reg <= q_a;  // <-- RDC path, UNSAFE
endmodule

```

RDC Coding Guidelines

Example — Crossing to a Non-Resettable Flop

- **pipe has no reset pin — async assertion still violates it.**
- NRR crossings are first-class RDC violations, same severity as domain-to-domain.

RDC - 07_rdc_to_non_resettable.v

```
// UNSAFE RDC to a non-resettable register (NRR).
// pipe has NO reset pin (pipeline / follower flop). When rst_a_n
// asserts, q_a changes asynchronously and pipe samples the changing
// value -> setup/hold violation on pipe -> metastability.
module rdc_to_non_resettable (
    output reg pipe,
    input  d_a,
    input  clk_a, clk_b,
    input  rst_a_n
);
    reg q_a;
    always @(posedge clk_a or negedge rst_a_n)
        if (!rst_a_n) q_a <= 1'b0;
        else          q_a <= d_a;

    always @(posedge clk_b)      // NRR: no reset
        pipe <= q_a;             // <-- RDC path, UNSAFE during assertion
endmodule

```

RDC Coding Guidelines

Example — Fixed by Reset Sequencing

- The b-domain synchronizer takes a_rst_n as its async reset input.
- **Result: b asserts no later than a, and releases only after a.**

RDC - 11_rdc_fix_reset_sequencing.v

```
// FIX 1: reset sequencing
// The Rx (b) domain reset is generated from the Tx (a) domain master reset:
//   assert : arst_n asserts -> a_rst_n AND b_rst_n assert together,
//            so Rx is in reset no later than Tx -> nothing to violate
//   release: b_rst_n is released only AFTER a_rst_n (ordered startup)
module rdc_fix_reset_sequencing (
    output a_rst_n, b_rst_n,
    input  clk_a, clk_b,
    input  arst_n       // primary async reset
);
    // a-domain reset synchronizer: released by clk_a
    reg arff1, arff2;
    always @(posedge clk_a or negedge arst_n)
        if (!arst_n) {arff2, arff1} <= 2'b00;
        else         {arff2, arff1} <= {arff1, 1'b1};
    assign a_rst_n = arff2;

    // b-domain synchronizer ASYNC INPUT comes from the a-domain master
    // reset -> sequenced removal, and the b domain is never out of
    // reset while the a domain resets (Rx-side safe).
    reg brff1, brff2;
    always @(posedge clk_b or negedge a_rst_n)
        if (!a_rst_n) {brff2, brff1} <= 2'b00;
        else          {brff2, brff1} <= {brff1, 1'b1};
    assign b_rst_n = brff2;
endmodule

```

RDC Coding Guidelines

Example — Fixed by Clock-Gate Isolation

- iso_n freezes the Rx clock across the Tx reset event — no edges, no violation.
- Glitch-free ICG: the latch only passes enable changes while clk_b is low.

RDC - 12_rdc_fix_clock_gate_isolation.v

```
// FIX 2: clock-gate isolation
// iso_n turns OFF the Rx clock before the Tx reset asserts.
// No clk_b edge -> no setup/hold sampling -> no metastability on b_reg.
// CONSTRAINT: iso_n must fall BEFORE tx_rst_n asserts
module rdc_fix_clock_gate_isolation (
    output reg b_reg,
    input  d_a,             // Tx-domain data
    input  clk_a, clk_b,
    input  tx_rst_n,        // Tx domain async reset
    input  rst_b_n,         // Rx domain async reset
    input  iso_n            // isolation enable, active-low
);
    // Tx flop
    reg tx_q;
    always @(posedge clk_a or negedge tx_rst_n)
        if (!tx_rst_n) tx_q <= 1'b0;
        else           tx_q <= d_a;

    // glitch-free ICG (latch is transparent only while clk_b is low)
    reg icg_latch;
    always @(*)
        if (!clk_b) icg_latch <= iso_n;
    wire gated_clk_b = clk_b & icg_latch;

    always @(posedge gated_clk_b or negedge rst_b_n)
        if (!rst_b_n) b_reg <= 1'b0;
        else          b_reg <= tx_q;   // safe while iso_n = 0
endmodule

```

RDC Coding Guidelines

Example — Fixed by Data Isolation

- iso_n forces d2 to 0 while the Tx domain resets — the Rx D-pin never moves.
- Constraint: iso_n must assert before tx_rst_n (proved by the RDC tool).

RDC - 13_rdc_fix_data_isolation.v

```v
// FIX 3: data isolation. Gate the Tx->Rx data path with an
// isolation enable so the Rx D-pin holds a stable value while the Tx
// domain resets. Same idea as UPF isolation, but sequenced by resets.
// CONSTRAINT: iso_n must assert BEFORE tx_rst_n asserts.
module rdc_fix_data_isolation (
    output reg b_reg,
    input  d_a,
    input  clk_a, clk_b,
    input  tx_rst_n, rst_b_n,
    input  iso_n        // isolation enable, active-low
);
    reg tx_q;
    always @(posedge clk_a or negedge tx_rst_n)
        if (!tx_rst_n) tx_q <= 1'b0;
        else           tx_q <= d_a;

    wire d2 = tx_q & iso_n;   // forced to 0 while isolated -> Rx D stable

    always @(posedge clk_b or negedge rst_b_n)
        if (!rst_b_n) b_reg <= 1'b0;
        else          b_reg <= d2; // safe during the Tx reset event
endmodule

```

RDC Coding Guidelines

Example — Soft Reset Done Right

- Soft reset generated and consumed in the SAME clock domain.
- **STA times the TxRstReg -> Tx -> Rx path as setup -> Caution, not Violation.**

RDC - 09_soft_reset_same_clock_safe.v

```v
// SAFE (IF STA times it): soft reset generated in the SAME clock
// domain as both the Tx flop and the Rx flop (DVCon Fig. 3b/3c case).
// * The assertion of soft_rst_n is synchronous to clk.
// * RDC tools report this as a CAUTION: STA must time the path
//   soft_rst_q -> tx_q (through the reset pin) -> rx.d as setup.
module soft_reset_same_clock_safe (
    output reg rx_reg,
    input  d,
    input  clk,
    input  rst_n                    // primary async reset
);
    reg soft_rst_n;                 // e.g. SW-controlled block reset
    reg tx_q;

    always @(posedge clk or negedge rst_n)
        if (!rst_n) soft_rst_n <= 1'b0;
        else        soft_rst_n <= 1'b1;

    always @(posedge clk or negedge soft_rst_n)
        if (!soft_rst_n) tx_q <= 1'b0;
        else             tx_q <= d;

    always @(posedge clk)
        rx_reg <= tx_q;             // safe in the SAME clock domain
endmodule

```

STA & RDC

STA & RDC

STA Checks for Reset Paths

- **Recovery / removal:** window checks on every async reset pin — report them explicitly.
- **Reset tree:** max transition and fanout like a clock net; time the full release path after layout.
- **Async assertion:** set_false_path from the reset port — untimed by design.
- **Soft-reset crossings:** a normal setup path — never false-path it.

- An RDC "Caution" is literally a to-do item in STA.

STA & RDC

SDC Snippets for Reset

- False-path the assertion, time the release, protect the tree from optimization.

RDC - RDC_STA_Constraints.tcl

```
1  ##############################################################################
2  ## RDC-related STA constraints (PrimeTime style)
3  ## Companion file for the RDC deck - see the "STA & RDC" section.
4  ##############################################################################
5
6  ## 1) Async reset ASSERTION is asynchronous by design -> do not time it.
7  ##    (Only when the reset reaches the flops directly from a port.)
8  set_false_path -from [get_ports arst_n]
9
10 ## 2) Reset RELEASE *is* timed: recovery/removal window checks run on
11 ##    every async reset pin of the synchronized reset trees.
12 report_constraint -all_violators -recovery -removal
13
14 ## 3) Same-clock soft-reset crossings (RDC "CAUTION" paths):
15 ##    the path TxRstReg/Q -> Tx reset pin -> Tx/Q -> Rx/D must be timed
16 ##    as a NORMAL setup path.
17 ##    NEVER false-path these - that would hide the soft-reset check:
18 # set_false_path -from [get_pins u_soft_rst_reg/Q]   ;# WRONG - do not do
19
20 ## 4) Protect the reset network from synthesis optimization
21 set_dont_touch_network [get_ports arst_n]
22 ## pre-CTS alternative while the reset tree is still ideal:
23 # set_ideal_network [get_nets rst_n]
24
25 ## 5) Reset-tree loads are as numerous as clock loads - constrain
26 ##    transition and fanout on reset nets like a clock net:
27 set_max_transition 0.800 [get_nets rst_n]
28 set_max_fanout 32 [get_ports arst_n]
29
```

Summary

Summary

Key Takeaways

- RDC = async reset ASSERTION crossing into another reset domain — the reset-release problem generalized.
- **Reset synchronizers fix reset REMOVAL;**
- **sequencing and isolation fix reset ASSERTION across domains.**
- Same async domain is not automatically safe (soft resets); different domains are not automatically broken (ordering).
- Close RDC with static checks + formal analysis + constraints; hand Cautions to STA.
- **Design it in early — RDC bugs found in silicon mean re-spins.**

References

References

Papers and Standards

[1] C. Cummings, D. Mills, "Asynchronous & Synchronous Resets? I am so confused! How will I ever know which to use?", SNUG San Jose 2002.

[2] Reetika, S. K. Khare (Siemens EDA), "Techniques to identify reset metastability issues due to soft resets", DVCon India 2023.

[3] W. Shi, Y. You, K. Takara, "Build Reliable and Efficient Reset Networks with a Comprehensive RDC Verification Solution", DVCon Europe 2020 (Accellera).

[4] Suggested: C. Cummings, "Clock Domain Crossing (CDC) Design & Verification Techniques Using SystemVerilog", SNUG Boston 2008.

[5] Suggested: C. Kwok, P. Viswanathan, P. Yeung, "Addressing the Challenges of Reset Verification in SoC Designs", DVCon US 2015.

FRIENDLY REMINDER

Israel has killed more than 13,000 children in Gaza since October 7 while others are suffering from severe malnutrition and do not “even have the energy to cry”,
says the United Nations Children’s Fund (UNICEF).

“Thousands more have been injured or we can’t even determine where they are. They may be stuck under rubble … We haven’t seen that rate of death among children in almost any other conflict in the world”
-UNICEF Executive Director

Save The Palestinian Children

Thanks